Vehicle-Agnostic Multi-Class Attack Attribution on the CAN Bus

Shuo Peng[1,a)] Zhihe Zhang[1,b)] Go Tsuruoka[1,c)] Tatsuya Mori[1,2,3,d)]

Abstract: Retrofitting authenticated messaging onto vehicles already in service is rarely practical, which leaves passive observation of the control bus as the realistic means of defense. Two properties of existing monitors restrict where they can be deployed. Most report only whether traffic is hostile, telling an operator neither the kind of manipulation nor the subsystem affected, and most consume either the arbitration identifier or the manufacturer's signal dictionary, neither of which transfers to a different vehicle program. The detector described in this paper declines both inputs. It reduces each sliding window of bus traffic to a statistical descriptor built from three quantities any passive listener can extract: the spacing between consecutive arrivals, the disorder of the payload bytes, and the number of payload bits altered between successive frames. Because the descriptor is an unordered aggregate, permuting the identifiers within a window leaves it invariant, and no signal dictionary is consulted at any stage. Discretizing the descriptor into one co-occurrence map per pair of quantities produces a compact tensor, which a small convolutional network assigns to one of fifteen categories. Macro-averaged F1 attains 0.97 on traffic recorded from a production vehicle in motion. Transferring the trained network to an unrelated vehicle requires labels for one fifth of the target windows, after which discrimination across that vehicle's six attack families averages 0.92 AUC; arrival spacing contributes several points to this figure although it is dispensable within the source vehicle. Forged frames that conform to a legitimate transmitter's schedule are caught at a recall above 0.99, whereas frame suppression remains invisible to an aggregate view. A hardware-in-the-loop platform reproduces the behavior under a third injection mechanism, and a single decision requires under ten milliseconds on one CPU core.

Keywords: automotive security, Controller Area Network, intrusion detection, attack attribution, cross-vehicle transfer

1. Introduction

A modern passenger vehicle coordinates braking, steering, and powertrain functions across dozens to over a hundred Electronic Control Units, and the Controller Area Network remains the dominant bus carrying that traffic[1]. CAN was specified for reliable delivery under electrical noise rather than for operation under an adversary. It authenticates no sender and encrypts no payload, so any node that gains write access to the bus can inject, replay, or suppress frames. External interfaces such as OBD-II ports, telematics modems, and infotainment stacks have repeatedly provided that access[2], [3].

Cryptographic message authentication addresses frame forgery at its root, and AUTOSAR Secure Onboard Communication specifies how to attach authentication data to CAN frames. Deploying it, however, requires key provisioning, per-frame bandwidth headroom, and cryptographic capability in every participating ECU. Those preconditions do not hold across the vehicles already on the road, which leaves a monitoring layer on the bus as the practical defense for legacy and partially secured platforms. A monitor, by contrast, can be added at a single observation point without touching any transmitting ECU[4].

Two properties limit how useful current bus-level detectors are once they leave the benchmark. The first concerns what the detector reports. Most published systems output a single benign-or-attack decision, which tells an operator that something is wrong but not what kind of manipulation occurred or which subsystem is affected. Attack-specific mitigation and post-incident forensics both need that second piece of information[4].

The second concerns what the detector needs in order to run. Many designs read the CAN identifier as a categorical input, train one predictor per identifier, or decode payload fields using the manufacturer's signal database, distributed as a DBC file. Identifier assignments and payload encodings are not standardized across platforms, so

[1] Graduate School of Fundamental Science and Engineering, Waseda University, Shinjuku, Tokyo 169–8555, Japan
[2] RIKEN AIP, Chuo, Tokyo 103–0027, Japan
[3] National Institute of Information and Communications Technology, Koganei, Tokyo 184–8795, Japan
[a)] housaku@nsl.cs.waseda.ac.jp
[b)] zzhihe@nsl.cs.waseda.ac.jp
[c)] go@nsl.cs.waseda.ac.jp
[d)] mori@nsl.cs.waseda.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

such a detector must be rebuilt for each new vehicle[5], [6]. The DBC dependence carries a second cost that is less often stated. A DBC file maps identifiers and payload fields onto vehicle functions, which is precisely the knowledge an adversary needs in order to locate safety-critical messages and craft targeted injections. Manufacturers therefore treat it as confidential[7]. Deploying a detector that requires the DBC means sharing the file with IDS vendors, aftermarket monitoring devices, and update infrastructure, widening the circle that holds it. A detector that does not require it can be developed and shipped while the file stays inside the manufacturer's existing trust boundary.

This paper describes TEV-IDS, a detector built to satisfy both concerns at once. It summarizes each traffic window by three quantities that can be computed from timestamps and payload bytes alone, and it treats detection as a 15-way attribution problem rather than a binary alarm. Section 2 distills the setting into three explicit requirements: identifier independence (R1), attribution rather than alarms (R2), and portability by calibration rather than redesign (R3). Our contributions are as follows.

- We define an identifier-free window representation and a rendering of it into three pairwise histograms, which together make the decision function invariant to any permutation of identifiers within a window (R1).
- We report, to our knowledge, the first 15-way attack-attribution result on CAN-MIRGU, a corpus recorded from a moving production vehicle, and compare against five replicated detectors retrained on the same label space (R2).
- We measure what it costs to move the detector to a second vehicle, and we separate the contribution of each feature axis to that transfer (R3).
- We report a negative result. Suspension attacks are invisible to a bus-aggregate view, and we explain why this is the structural price of R1 rather than a consequence of insufficient training.

Section 2 derives R1–R3 and compares existing detectors against them; Section 3 constructs the detector; Section 4 fixes the protocol and corpora; Section 5 answers five questions, each tied to the requirement it tests; and Section 6 discusses the blind spot, the limitations, and future work before Section 7 concludes.

2. Problem Setting

2.1 Threat Model

We assume an adversary with write access to a CAN segment, gained through a compromised ECU, a malicious OBD-II dongle, or a wireless path into a gateway. The adversary can inject new frames, replay previously observed ones, and forge frames carrying the identifier of a legitimate sender. We further allow the adversary to time injections so that they fall inside the transmission schedule of the target ECU. Such timing defines a masquerade attack and defeats frequency-based monitors[8].

We assume the adversary cannot modify the detector, which we deploy as a passive listener on the bus or at a gateway tap. Physical-layer defenses that fingerprint senders from analog voltage characteristics[9] are outside our scope, since they require instrumentation that a software monitor does not have.

2.2 Deployment Constraints

Three constraints shape the design. The detector must finish each decision within 125 ms, the interval at which its output is consumed (the latency constraint). It must run on a gateway-class processor without a GPU (the hardware constraint). And it must be portable to a vehicle whose identifier assignments and payload encodings were never seen during training, using only the labeled traffic that a supplier could plausibly collect on a test bench (the portability constraint). Section 5.5 tests the first two; the third, together with the reporting concern of Section 1, induces the requirements below.

2.3 Design Requirements and Related Detectors

The constraints map onto three requirements, against which we also position existing detectors.

R1: the decision must not depend on identifier semantics. The largest family of CAN detectors consumes the identifier stream directly. DCNN[10] rasterizes sequences of consecutive arbitration identifiers into binary images and classifies them with a deep convolutional network; GIDS[11] trains a generative adversarial network on similar identifier images so that its discriminator doubles as the detector; Rec-CNN[12] converts identifier sequences into recurrence plots before convolution; and LSTM-based designs model identifier sequences with recurrent networks[13]. Graph-based detectors make the identifier the node set itself: GGNB[14] extracts features from per-window identifier graphs for a Gaussian naive Bayes classifier, and StatGraph[15] learns over multi-view statistical graphs of identifier transitions. All of these bind the learned function to one vehicle's identifier assignment: the same physical manipulation renders as a different image, sequence, or graph on a vehicle whose DBC assigns different identifiers, so the model must be rebuilt rather than recalibrated. Signal-level detectors avoid the raw identifier value but pay with the DBC itself. CANet[16] instantiates one recurrent input branch per identifier and sizes it from the signal definitions; CANShield[17] decodes payloads into physical signals before feeding a multi-scale autoencoder ensemble; X-CANIDS[18] likewise decodes DBC signals to explain its alerts; and AMICA[19] routes per-identifier streams through an attention model. Per-identifier architecture and signal decoding both presume exactly the file whose confidentiality Section 1 argued should stay inside the manufacturer. Neither family satisfies R1.

R2: the output must name the attack, not merely flag it. Reported results in this literature are predominantly binary[4]: a single benign-or-attack deci-

© 2026 Information Processing Society of Japan

sion, or one detector per attack whose scores are averaged, evaluated on benchmarks that expose four attack types — denial of service, fuzzing, and two spoofing variants. Either style leaves the operator to work out after the alarm what is being manipulated, which is the information that mitigation and forensics actually consume. A macro-averaged score over a taxonomy of fifteen classes measures a different and harder capability: the model must separate not only attack from benign but also attack families that perturb the bus in similar ways, and it is penalized equally for every class regardless of support. Published binary figures and attribution figures are therefore not comparable; Table 2 makes the gap concrete by retraining published detectors for attribution.

R3: a change of vehicle must be a calibration step, not a redesign. Evaluation in this area rarely leaves the training vehicle: the common protocol trains and tests on captures from a single car, which cannot reveal a representation that has silently memorized that car's identifier set or payload encodings. An independent re-evaluation[20] retrained detectors that reach near-perfect F1 on the four-attack benchmark and found substantially worse performance on the richer taxonomy of ROAD[6], which suggests that in-domain scores overstate portability. The two families above cannot do better by construction: identifier-based models face a new input space and must be retrained, and signal-level models cannot run without the target's DBC. We therefore treat the vehicle change as an explicit experimental condition: Section 3.4 fixes a calibration procedure that alters no design choice, and Section 5.3 measures its cost in labeled target windows.

To our knowledge, no published CAN detector has been evaluated against all three requirements simultaneously. The remainder of the paper constructs one and reports where it succeeds and where it structurally cannot.

3. Detector Construction

Figure 1 shows the four stages. The identifier field is discarded at the first stage and never re-enters the pipeline.

3.1 Window Abstraction

The bus is read as a stream of frames $m_i = (t_i, d_i)$ carrying a timestamp and a payload, with the identifier removed. A single frame cannot separate a gradual payload manipulation from ordinary signal movement, so we cut the stream into overlapping windows of duration $\tau = 250\,\text{ms}$ advanced by a stride of $s = 125\,\text{ms}$.

The window duration follows from two opposing requirements rather than from tuning. At 500 kbps a standard CAN 2.0A frame with an eight-byte payload occupies 0.22 to 0.27 ms, so at bus loads of 30 to 70% a 250 ms window accumulates several hundred frames, which is enough for the entropy and variation statistics below to be stable. A longer window blurs a short burst into the surrounding benign traffic and delays the decision beyond the latency budget. The 50% overlap guarantees that any attack episode falls entirely inside at least one window, and the stride equals the 125 ms decision interval of Section 2.2, so the pipeline emits exactly one decision per consumption interval.

3.2 The Three Axes

Each frame is mapped to a triple $v_i = [T_i, E_i, V_i]$.

The *temporal* coordinate is the logarithmic inter-arrival interval in the global frame stream,

$$T_i = \ln(1 + (t_i - t_{i-1}))\,. \tag{1}$$

We compute it over the aggregate stream rather than per identifier, so it responds to changes in total bus load, which flooding and aggressive injection produce. The logarithm compresses the wide dynamic range of CAN inter-arrival times so that small deviations remain visible. The statistic is deliberately blind to any single transmitter's schedule; Section 6.1 examines the cost of that blindness.

The *entropy* coordinate is the Shannon entropy of the byte distribution within the payload,

$$E_i = -\sum_{x \in \mathcal{U}} P(x) \log_2 P(x), \tag{2}$$

where $P(x)$ is the empirical frequency of byte value x in d_i. Payloads encoding physical quantities occupy constrained ranges and repeat, so their entropy is low and stable. Random or malformed content raises it. With at most eight payload bytes, $E_i \le \log_2 L \le 3$ bits; benign frames stay well below the bound, and fuzzed content approaches it. The statistic needs no knowledge of what the bytes mean, which is what keeps the construction DBC-free.

The *variation* coordinate is the Hamming distance between consecutive payloads,

$$V_i = \sum_{b=1}^{L} \mathbf{1}\left[d_i^{(b)} \neq d_{i-1}^{(b)}\right], \tag{3}$$

counting bit positions that changed, so $V_i \in [0, 8L]$. Physical state evolves smoothly, so benign traffic produces small and regular values, whereas replayed or counter-manipulated payloads produce abrupt jumps that timing and entropy alone do not expose. The mechanism differs by attack: a replayed segment is discontinuous with the live signal at its splice points, and a spoofed constant freezes bits that benign encoders keep moving, so both register even when every individual payload looks plausible.

3.3 Rendering to Pairwise Histograms

Classifying the three-dimensional point cloud directly is expensive, so each window is projected onto the three coordinate planes, temporal-entropy, temporal-variation, and entropy-variation. Each projection is accumulated into an $R \times R$ two-dimensional histogram with $R = 96$, counts are compressed by $\log(1+n)$ so that sparse structure stays visible, and the three planes are stacked into one $96 \times 96 \times 3$ tensor. The resolution balances two failure modes: a finer

© 2026 Information Processing Society of Japan

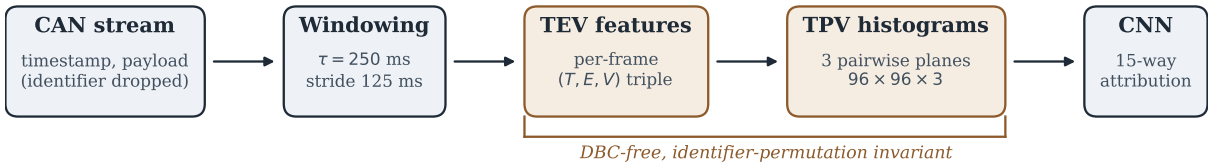


Fig. 1 Stages of TEV-IDS. Timestamps and payload bytes enter the feature construction; the identifier field is dropped at ingest.

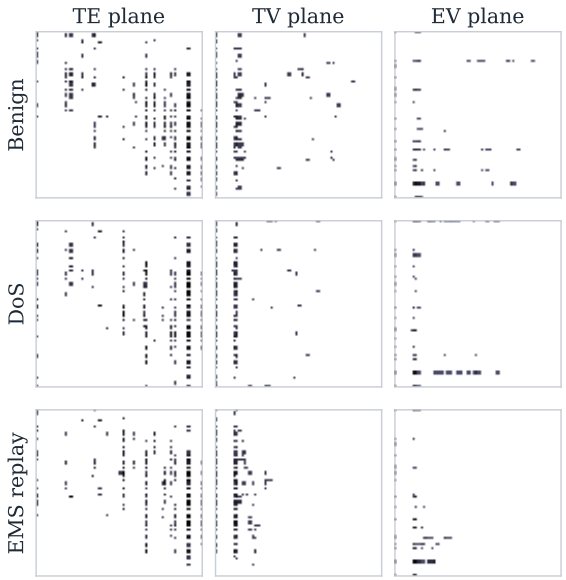


Fig. 2 Individual projection planes for three window types, log-scaled. Columns are the three pairwise planes; rows are one benign window, a DoS window, and a replay window.

grid scatters a window's few hundred points into isolated cells, and a coarser one merges attack mass into benign ridges.

The rendering makes the identifier invariance concrete. A histogram is a multiset summary, so permuting which identifier carried which frame inside a window leaves the tensor unchanged. Projection ranges are estimated from benign traffic of whichever corpus is being processed, so a target vehicle with a different ambient rate does not shift the grid.

Figure 2 shows the three planes separately for one benign window and two attack windows. DoS concentrates mass in the low-interval region of both planes involving the temporal axis. The replay window keeps a benign-looking temporal profile and separates instead on the entropy-variation plane — the reason the third plane exists.

3.4 Classifier and Cross-Vehicle Calibration

A small CNN classifies the tensor: two convolutional blocks of 32 and 64 filters with 3×3 kernels, each followed by 2×2 max pooling, then a 128-unit dense layer with dropout 0.5 and a softmax over the label set. The model holds 3.99 M parameters and one forward pass costs 97.85 MFLOPs. The size targets the hardware constraint of Section 2.2; Section 5.5 reports the resulting CPU latency. Training uses focal loss with $\gamma = 2$ for 30 epochs at batch 64, with Adam at a peak learning rate of 10^{-3}.

Table 1 Configuration used throughout the evaluation

Item	Setting
Window / stride	250 ms / 125 ms
Histogram grid R	96 per plane, 3 planes
Backbone	Conv 32/64, Dense 128, dropout 0.5
Loss / epochs / batch	focal ($\gamma = 2$) / 30 / 64
Source split	per-file 70/15/15, chronological
Target split	contiguous blocks, boundary dropped
Hardware	RTX 4000 Ada, 1-thread CPU

Moving to another vehicle changes the ambient traffic statistics but not the geometry of the representation, so we adapt rather than retrain. Let $\mathcal{D}_{\text{tgt}}$ be labeled windows from the target vehicle. We draw a stratified calibration subset with $|\mathcal{D}_{\text{cal}}| = \alpha|\mathcal{D}_{\text{tgt}}|$, spread over attack scenarios and ambient traffic, and reserve the remainder for testing. Starting from the source weights, all layers are fine-tuned on $\mathcal{D}_{\text{cal}}$ with cosine annealing from 10^{-6} to 10^{-4} and early stopping on an internal split. Fine-tuning every layer lets the network re-center the benign manifold of the new vehicle, while the low peak rate keeps it from discarding source structure. Nothing else changes: no feature definition, no histogram-range logic, no hyper-parameter of Table 1.

4. Evaluation Protocol

Table 1 lists the settings. Two public corpora are used, joined in Section 5.5 by a hardware-in-the-loop platform.

CAN-MIRGU[21] is the source domain. It was recorded from a production passenger vehicle, with benign traffic collected over several days of real driving and attack captures taken while the vehicle was in motion rather than on a bench, which distinguishes it from earlier collections. The injection captures cover DoS, fuzzing, powertrain (EMS) manipulation and replay, speedometer, steering and power-steering, brake, gear, drive-mode, and warning-light (door-open and forward-collision) targets. Many captures are sub-variants of one manipulation that differ only in which identifier is targeted; our representation cannot distinguish such sub-variants even in principle, so we merge them, consolidating the release into 15 classes: benign plus fourteen attack families. Figure 3 reports their test-set supports. The release additionally ships masquerade and suspension captures synthesized from real traces; these are held out of the 15-class space entirely and reserved for the targeted evaluations of Sections 5.4 and 6.1.

ROAD[6] is the target domain. It was collected at Oak Ridge National Laboratory from a different production vehicle exercised on a dynamometer, and its attacks are physically verified: each injection demonstrably produced the

© 2026 Information Processing Society of Japan

intended effect on the vehicle. We draw the benign pool for the calibration procedure of Section 3.4 from its ambient recordings and group its attack captures into six categories over which we compute the transfer metric. The corpora share nothing: vehicle, identifier assignments, payload encodings, driving condition, and capture instrumentation all differ. The pair is therefore a genuine test of R3.

Splits are designed around the 50% window overlap. A random split would place near-duplicate windows, sharing half their frames, on both sides of the train/test boundary and quietly inflate every reported number. Source-side splits are therefore chronological within each capture (70/15/15 per file). On the target side, each capture is divided into contiguous blocks assigned wholly to calibration or test, and windows straddling a block boundary are discarded, so no frame contributes to both sides.

Source results are macro-F1 over 15 classes; target results are macro-AUC over the six ROAD categories. Macro averaging is forced by the label distribution: benign accounts for 69,175 of the 71,694 test windows (96.5%), so a classifier that never raises an alarm already reports 0.965 accuracy, and errors on families with under a hundred test windows barely move a micro-averaged figure. For the same reason Table 2 reports macro-F1 only.

5. Results

5.1 Does an Identifier-Free Representation Support 15-Way Attribution? (R1, R2)

This experiment tests R1 and R2 jointly: whether a representation that never observes the identifier carries enough information to name the attack among fifteen alternatives. TEV-IDS reaches a macro-F1 of 0.9684 on the held-out CAN-MIRGU test set, averaged over five training runs. To our knowledge this is the first fine-grained multi-class result reported on this corpus. Prior work targets binary or per-attack detection on older benchmarks.

Figure 3 shows every class clearing 0.90. The weakest entries are EMS replay (long) and gear shifter, which the confusion pattern places against each other and against speedometer traffic. These are the classes whose sub-variants were merged, so a residual overlap after merging is the expected outcome. The classes with the smallest support are not the weakest ones, which indicates the focal objective is holding the minority classes up.

Table 2 compares against five external detectors, re-implemented and retrained on the same 15-class windows and splits with their published hyper-parameters. The margin over the strongest of them is 6.0 points of macro-F1. The ordering is informative. AMICA-lite comes closest: its transformer over per-identifier streams retains full payload evidence, at the price of an architecture bound to the source vehicle's identifier set. HistCAN, the next closest, is likewise built on the identifier stream alone, one-hot identifier images fused with their recent history. DCNN loses the most ground, and it is the design that depends most directly on the identifier: once sub-variants that target different identifiers are merged into one class, the near one-to-one mapping between class and identifier that it exploits no longer exists. Rec-CNN compresses the same identifier sequence into recurrence plots and falls to 0.5056. CANShield is included for completeness but is an anomaly detector forced into a 15-way task, so its figure should be read as a statement about task mismatch rather than about the method. In R1's terms, discarding the identifier costs nothing against these baselines: the two closest competitors both read it yet trail by six points or more.

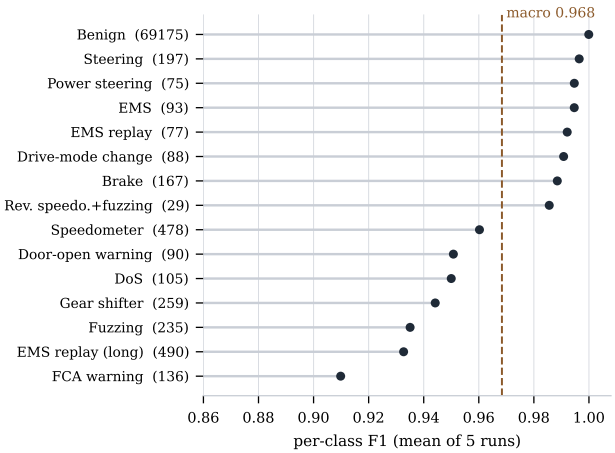


Fig. 3 Per-class F1 on the CAN-MIRGU test set, sorted. Test-set support in parentheses.

Table 2 Comparison against replicated detectors, 15-class CAN-MIRGU

Method	Basis	Macro-F1
TEV-IDS	TEV hist. + CNN	**0.9684**
AMICA-lite[19]	per-ID Transf.	0.9082
HistCAN[22]	hist.-fusion CNN	0.8611
DCNN[10]	CAN-ID image	0.7738
Rec-CNN[12]	recurrence plot	0.5056
CANShield[17]	multi-scale AE	0.1404

5.2 Which Axes Carry the Discriminative Load? (R1)

R1 fixes the feature pool to the three identifier-free axes; this experiment asks which of them earn their place. Retraining on the three pairwise planes in isolation separates their roles. The entropy-variation pair alone reaches 0.9700, statistically indistinguishable from the full three-channel tensor at 0.9709. Dropping entropy collapses the model to 0.5859, and the temporal-entropy pair recovers to 0.8109.

Most CAN-MIRGU attack families are payload manipulations delivered at unremarkable rates. The temporal planes therefore separate flooding from everything else but carry little between-family information; entropy responds to what the injected bytes are, variation to how they move, and that is where fourteen families differ.

Entropy is therefore the load-bearing axis in the source domain, variation is strongly complementary to it, and timing contributes nothing measurable while the vehicle is held fixed. That last observation is worth stating plainly,

© 2026 Information Processing Society of Japan

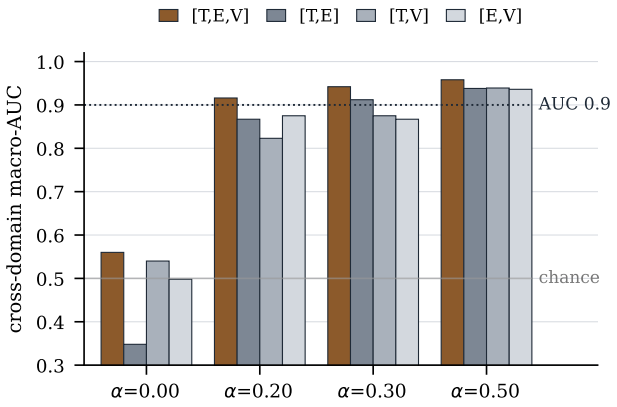


Fig. 4 Cross-domain macro-AUC on ROAD against calibration budget, for the full tensor and the three pairwise planes.

because it reverses in the next experiment.

5.3 What Does a Change of Vehicle Cost? (R3)

R3 demands that the vehicle change be a calibration step, not a redesign. The redesign half holds by construction — architecture, features, and ranges are frozen — so what remains is to measure the calibration half. Figure 4 sweeps that budget on ROAD. Without target labels every configuration sits near the chance line, so a detector shipped to a new vehicle with no calibration is not usable. The first 20% of labels buys 17.8 points of AUC per 10% of budget, the next 10% buys 2.6 points, and the final 20% buys 0.8 points per 10%. We therefore operate at $\alpha = 0.20$, which reaches 0.916 and clears the conventional 0.9 threshold for outstanding discrimination at 40% of the labeling cost of $\alpha = 0.50$.

The timing axis changes role here. In-domain it was redundant; across vehicles the full tensor stays above the entropy-variation pair at every budget, by 4.1 points at $\alpha = 0.20$ and 7.4 points at $\alpha = 0.30$. Inspecting the six categories individually locates the reason. Fuzzing is the weakest category for the entropy-variation pair, because ROAD fuzzing payloads mimic the target vehicle's own byte-change and entropy statistics closely enough that the payload planes cannot separate them. Adding the timing axis lifts that category substantially. The recovery is not yet dependable, since the spread across block splits on this category is wide, but the direction is consistent.

Calibration thus replaces redesign; the cost is labels for one fifth of the target windows, and Section 6.2 examines who realistically bears it.

5.4 Are Schedule-Conforming Attacks Caught? (R1)

Masquerade is the stress test for the feature pool R1 permits: the attack is defined by leaving nothing for a schedule monitor to see, so whatever catches it must live in the payload axes. CAN-MIRGU ships masquerade and suspension captures synthesized from real traces. Masquerade removes the legitimate frame preceding each spoofed frame, leaving a stream whose timing looks scheduled. We evaluate them with the trained 15-class model without retraining, flagging a window whenever the argmax is any class other than benign.

Masquerade windows are caught with a recall of 0.9962. This is the case that motivates the payload axes: the timing channel sees nothing unusual, and the detection rests on the entropy and variation planes, whose signature is unchanged from the sibling injection capture. Suspension is a different story and is treated in Section 6.

5.5 Does It Run Inside the Decision Window?

This experiment tests the latency budget of Section 2.2. Under a single-thread CPU benchmark over 50,000 windows the forward pass has a mean latency of 3.34 ms and a 99th percentile of 6.00 ms, against the 125 ms budget, giving a zero deadline-miss rate. The delay from the first injected frame to the close of the first window containing it is 125 to 250 ms plus inference, so roughly 130 to 255 ms end to end. That sits well inside the 0.7 to 1.5 s range usually cited for driver brake reaction.

We further deployed the unmodified pipeline on the PASTA testbed[23], an open hardware-in-the-loop platform built around white-box ECUs. Three domain ECUs, standing in for powertrain, chassis, and body functions, exchange frames with a central gateway over physical CAN wiring, and a vehicle simulator drives the assembly through a bridge, so arbitration, bit timing, and error handling come from real transceivers, not a replayed log. It contributes two things. First, a third injection regime: CAN-MIRGU's attacks were injected into a vehicle in motion, ROAD's were captured on a dynamometer, and PASTA's originate from a node on the same physical segment as the ECUs, so agreement across the three argues the pipeline is not fitted to one capture setup. Second, it permits experiments unsafe on a real vehicle, such as a sustained flood at the highest rate the bus accepts.

On a seven-class attribution task defined over the scenarios staged on the platform, the pipeline reaches a macro-F1 of 0.9993 with no protocol change: the same windowing, features, rendering, and architecture, retrained on PASTA captures (the PASTA-trained row of Table 3). A live session of 4,823 consecutive windows — roughly ten minutes of continuous operation — spanned ambient traffic, spoofing, and a 1 ms-period flood that raised the load to 417 frames per window; the worst window completed in 20.9 ms, a factor of six inside the budget under the heaviest load we could generate.

False alarms on benign-only traffic are given in Table 3. The source model raises none across 69,175 in-domain benign windows. The ROAD figure is measured on the uncalibrated model evaluated zero-shot and therefore upper-bounds the deployed configuration, which applies the $\alpha = 0.20$ calibration first. The practical concern is alarm fatigue: at eight windows per second, the zero-shot 1.07% would mean several alarms per minute; an uncalibrated deployment is not viable.

© 2026 Information Processing Society of Japan

Table 3 False-alarm rate on benign-only traces

Trace (model)	Windows	FPR
CAN-MIRGU test (source)	69,175	0.00%
ROAD ambient (zero-shot)	19,660	1.07%
PASTA test (PASTA-trained)	1,082	0.09%

6. Discussion

The evaluation leaves one attack family completely uncovered, and the reason deserves more than a caveat. We examine that failure and argue it is structural (Section 6.1), then state the limitations of the study (Section 6.2) and the work they set up (Section 6.3).

6.1 A Structural Blind Spot: Suspension Attacks

Suspension attacks are not detected at all. Across five target identifiers, none of the 4,185 suspension windows is flagged, and no threshold adjustment recovers them. Together with Section 5.4, the result defines the blind spot sharply. An adversary who adds or alters frames leaves payload-statistical evidence that survives even a perfect transmission schedule, and is caught at 0.996 recall. An adversary who only removes frames leaves no such evidence, and is never caught.

A suspension capture is a benign trace with all frames of one low-rate identifier deleted over an interval, relabeled as attack. Deleting a low-rate identifier introduces no anomalous payload, so the entropy and variation planes see benign content, and it shifts the aggregate inter-arrival distribution by an amount too small to register, so the temporal plane sees benign timing. The classifier is handed benign traffic under a different label and reads it as benign, which is the correct response to what it was shown. Detecting deleted frames requires a per-identifier expectation of what should have arrived, and that is exactly the state the design gives up in order to satisfy R1. The trade is structural: identifier invariance buys portability and costs the ability to notice a missing sender.

The cure does not require abandoning R1 wholesale. A per-identifier liveness monitor needs only the observed identifier set and the empirical period of each, both learnable from minutes of ambient traffic without a DBC; it reads identifier identity, never meaning. A timeout table over expected periods would flag a silenced transmitter within a few missed slots at negligible cost, and pairing it with the identifier-free classifier covers the absence case while leaving the portability of Sections 3 and 5 intact; Section 6.3 returns to this.

6.2 Limitations

Six limitations bound the claims. (i) The cross-vehicle protocol consumes labeled attack traffic from the target vehicle: $\alpha = 0.20$ presumes an actor — realistically an OEM or tier-one supplier — able to stage attacks on a bench; the protocol reduces the cost of a vehicle change but does not remove it. (ii) Both corpora and the testbed speak Classical CAN with 11-bit identifiers and payloads of at most eight bytes; CAN-FD's 64-byte payloads change the entropy and variation statistics, and Automotive Ethernet changes the framing entirely, so nothing measured here transfers to either without new experiments. (iii) The three axes are transparent scalars that an adversary who knows the detector could try to shape traffic to preserve; we have not evaluated such an adaptive adversary. (iv) The masquerade and suspension captures are synthesized from real traces by the corpus authors rather than executed live: accepted practice for attacks unsafe to run on a moving vehicle, but one step removed from a compromised ECU falling silent in traffic. (v) The transfer evidence is one source and one target vehicle: two platforms demonstrate that transfer is possible and measure its price, but they do not establish how that price varies across the vehicle population. (vi) Attribution is issued at window granularity: the detector names the attack family, not the offending identifier or node, so localization for response remains a separate step.

6.3 Future Work

Five directions follow. The most direct is the hybrid detector sketched above: the identifier-free classifier for attribution, paired with a per-identifier liveness table for absence, evaluated against the same suspension captures that defeat the aggregate view.

A second experiment would price R1 from the other side. Augmenting the TEV representation with an identifier-derived channel should recover some in-domain headroom and, we expect, surrender it in transfer, since the added channel is exactly the vehicle-specific structure calibration cannot cheaply re-learn; measuring both halves would give the cost of identifier independence in points of macro-F1 and macro-AUC.

Third, the 20% label requirement is the main obstacle to deployment. Two routes suggest themselves: synthetic calibration — injecting frames offline into the target's own ambient logs, the mechanism by which the corpora construct their masquerade captures — and unsupervised adaptation on ambient traffic alone; whether the latter can lift the chance-level zero-shot AUC of Figure 4 is open.

Fourth, protocol scope and adversarial robustness: extending to CAN-FD is largely re-measurement — the definitions of T, E, and V survive, but ranges must be re-estimated for 64-byte frames — whereas Automotive Ethernet replaces the ingest layer; and a mimicry study should ask what an attacker constrained to preserve all three axis distributions can still accomplish; the constraint and the attack goal are in tension, but the size of the feasible region is an empirical question.

Finally, the latency figures of Section 5.5 come from a workstation core; a port to an automotive-grade gateway SoC would turn the budget claim into a measurement on target silicon.

© 2026 Information Processing Society of Japan

7. Concluding Remarks

We described a vehicle-agnostic CAN intrusion detector that meets the three requirements of Section 2. Its feature construction never reads the identifier field and never needs the manufacturer's signal database (R1); it reports which of fifteen attack categories it is seeing rather than a binary alarm (R2), reaching a macro-F1 of 0.9684 in domain; and it treats a change of vehicle as calibration rather than redesign (R3), transferring to a second production vehicle at a macro-AUC of 0.92 with labels for a fifth of the target windows. It catches schedule-conforming masquerade at 0.996 recall and runs inside the 125 ms decision window on a single CPU core.

The two findings we consider most useful to report are the ones that cut against a simple story. The timing axis is redundant in domain and matters across vehicles, so an ablation run on one vehicle would have argued for removing the feature that carries the transfer. And suspension attacks are structurally invisible to a bus-aggregate view, which bounds what identifier invariance can deliver and points at the hybrid design of Section 6.3 as the way to cover it.

Acknowledgments This work was partially supported by JST CREST JPMJCR23M4, JST SPRING JPMJBS2429 and JST BOOST, Japan Grant Number JPMJBS2429.

References

[1] Bozdal, M., Samie, M., Aslam, S. and Jennions, I.: Evaluation of CAN Bus Security Challenges, *Sensors*, Vol. 20, No. 8, p. 2364 (2020).

[2] Checkoway, S. et al.: Comprehensive Experimental Analyses of Automotive Attack Surfaces, *Proc. USENIX Security Symposium* (2011).

[3] Miller, C. and Valasek, C.: Remote Exploitation of an Unaltered Passenger Vehicle, *Black Hat USA* (2015).

[4] Lampe, B. and Meng, W.: Intrusion Detection in the Automotive Domain: A Comprehensive Review, *IEEE Communications Surveys & Tutorials*, Vol. 25, No. 4, pp. 2356–2426 (2023).

[5] Pawelec, K., Bridges, R. A. and Combs, F. L.: Towards a CAN IDS Based on a Neural Network Data Field Predictor, *Proc. ACM Workshop on Automotive Cybersecurity*, pp. 31–34 (2019).

[6] Verma, M. E. et al.: A Comprehensive Guide to CAN IDS Data and Introduction of the ROAD Dataset, *PLOS ONE*, Vol. 19, No. 1, p. e0296879 (2024).

[7] Buscemi, A., Turcanu, I., Castignani, G., Panchenko, A., Engel, T. and Shin, K. G.: A Survey on Controller Area Network Reverse Engineering, *IEEE Communications Surveys & Tutorials*, Vol. 25, No. 3, pp. 1445–1481 (2023).

[8] Moriano, P., Bridges, R. A. and Iannacone, M. D.: Detecting CAN Masquerade Attacks with Signal Clustering Similarity, *Workshop on Automotive and Autonomous Vehicle Security (AutoSec)* (2022).

[9] Choi, W., Joo, K., Jo, H. J., Park, M. C. and Lee, D. H.: VoltageIDS: Low-Level Communication Characteristics for Automotive Intrusion Detection System, *IEEE Trans. Information Forensics and Security*, Vol. 13, No. 8, pp. 2114–2129 (2018).

[10] Song, H. M., Woo, J. and Kim, H. K.: In-Vehicle Network Intrusion Detection Using Deep Convolutional Neural Network, *Vehicular Communications*, Vol. 21, p. 100198 (2020).

[11] Seo, E., Song, H. M. and Kim, H. K.: GIDS: GAN Based Intrusion Detection System for In-Vehicle Network, *Proc. Annual Conference on Privacy, Security and Trust (PST)*, pp. 1–6 (2018).

[12] Desta, A. K., Ohira, S., Arai, I. and Fujikawa, K.: Rec-CNN: In-Vehicle Networks Intrusion Detection Using Convolutional Neural Networks Trained on Recurrence Plots, *Vehicular Communications*, Vol. 35, p. 100470 (2022).

[13] Hossain, M. D., Inoue, H., Ochiai, H., Fall, D. and Kadobayashi, Y.: LSTM-Based Intrusion Detection System for In-Vehicle CAN Bus Communications, *IEEE Access*, Vol. 8, pp. 185489–185502 (2020).

[14] Islam, R., Devnath, M. K., Samad, M. D. and Al Kadry, S. M. J.: GGNB: Graph-Based Gaussian Naive Bayes Intrusion Detection System for CAN Bus, *Vehicular Communications*, Vol. 33, p. 100442 (2022).

[15] Wang, K., Jiang, Q., Wang, B., Wu, Y. and Zhang, H.: StatGraph: Effective In-Vehicle Intrusion Detection via Multi-View Statistical Graph Learning, *IEEE Trans. Mobile Computing*, Vol. 25, No. 5, pp. 6335–6351 (2026).

[16] Hanselmann, M., Strauss, T., Dormann, K. and Ulmer, H.: CANet: An Unsupervised Intrusion Detection System for High Dimensional CAN Bus Data, *IEEE Access*, Vol. 8, pp. 58194–58205 (2020).

[17] Shahriar, M. H., Xiao, Y., Moriano, P., Lou, W. and Hou, Y. T.: CANShield: Deep-Learning-Based Intrusion Detection Framework for Controller Area Networks at the Signal Level, *IEEE Internet of Things Journal*, Vol. 10, No. 24, pp. 22111–22127 (2023).

[18] Jeong, S., Lee, S., Lee, H. and Kim, H. K.: X-CANIDS: Signal-Aware Explainable Intrusion Detection System for Controller Area Network-Based In-Vehicle Network, *IEEE Trans. Vehicular Technology*, Vol. 73, No. 3, pp. 3230–3246 (2024).

[19] Alkhatib, N., Achaji, L., Mushtaq, M., Ghauch, H. and Danger, J.-L.: WIP: AMICA: Attention-Based Multi-Identifier Model for Asynchronous Intrusion Detection on Controller Area Networks, *Symposium on Vehicle Security and Privacy (VehicleSec)* (2023).

[20] Guerra, L., Xu, L., Bellavista, P., Chapuis, T., Duc, G., Mozharovskyi, P. and Nguyen, V.-T.: AI-Driven Intrusion Detection Systems (IDS) on the ROAD Dataset: A Comparative Analysis for Automotive Controller Area Network (CAN), *Proc. Cyber Security in CarS Workshop (CSCS '24)*, ACM, pp. 39–49 (2024).

[21] Rajapaksha, S., Madzudzo, G., Kalutarage, H., Petrovski, A. and Al-Kadri, M. O.: CAN-MIRGU: A Comprehensive CAN Bus Attack Dataset from Moving Vehicles for Intrusion Detection System Evaluation, *Symposium on Vehicle Security and Privacy (VehicleSec)* (2024).

[22] Zhuo, S., Li, N. and Ren, K.: HistCAN: A Real-Time CAN IDS with Enhanced Historical Traffic Learning Capability, *Symposium on Vehicle Security and Privacy (VehicleSec)* (2024).

[23] Toyama, T., Yoshida, T., Oguma, H. and Matsumoto, T.: PASTA: Portable Automotive Security Testbed with Adaptability, *Black Hat Europe*, London, UK (2018).

© 2026 Information Processing Society of Japan